UNIT-5

LECTURE-1

Key management and distribution:

From the view of developers, apply cryptographic or encryption is just import a crypto library and then follow documentation, best practices or just copy a snippet code from Stack overflow but they lack of understanding the fundamental of principles and mechanisms. That can lead to misuse of cryptography and make vulnerable for their systems. This article as a part of Cryptography 101 series I write as a note for myself and for anyone who may need a quick overview about key management and distribution.

Learning Objectives

- Concept of a key hierarchy.
- Understand the issues involved in using asymmetric encryption to distribute symmetric keys.
- Present an overview of approaches to public-key distribution and analyze the risks involved in various approaches.
- List and explain the element in a X.509 certificate.
- Present an overview of public-key infrastructure concepts.

The topics of cryptographic key management and cryptographic key distribution are complex, involving **cryptographic**, **protocol**, and **management considerations**.

KEY POINTS

- Key distribution is the function that delivers a key to two parties who wish to exchange secure encrypted data. Some sort of mechanism or protocol is needed to provide for the secure distribution of keys.
- Key distribution often involves the use of master keys, which are infrequently used and are long lasting, and session keys, which are generated and distributed for temporary use between two parties.
- Public-key encryption schemes are secure only if the authenticity of the public key is assured. A public-key certificate scheme provides the necessary security.
- X.509 defines the format for public-key certificates. This format is widely used in a variety of applications.
- A public-key infrastructure (PKI) is defined as the set of hardware, software, people, policies, and procedures needed to create, manage, store, distribute, and revoke digital certificates based on asymmetric cryptography.
- Typically, PKI implementations make use of X.509 certificates.

The topics of cryptographic key management and cryptographic key dist ibutionarecomplex, involving cryptographic, protocol, and management c onsiderations. The purpose of this chapter is to give the reader a feel for t he issues involved and a broad survey of the various aspects of key mana gement and distribution. For more information, the place to start is the t hree-volume NIST SP 800-57,

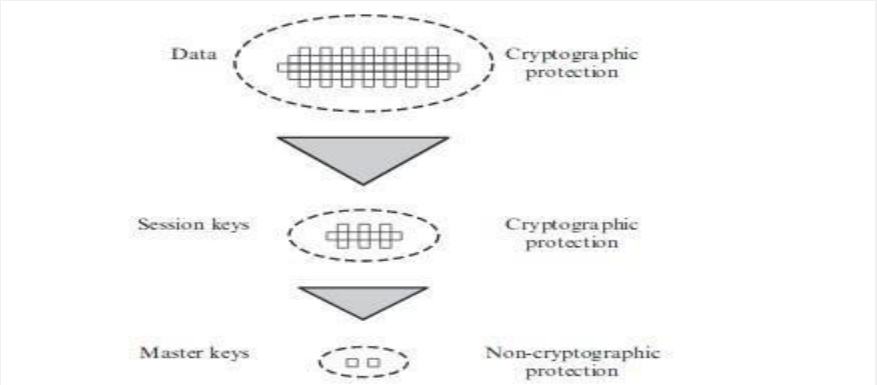
LECTURE-2

Symmetric key distribution using symmetric encryption:

For symmetric encryption to work, the two parties to an exchange must share the same key, and that key must be protected from access by others.

For two parties A and B, key distribution can be achieved in a number of ways, as fllows:

1. A can select a key and physically deliver it to B
2. A third party can select the key and physically deliver it to A and B
3. If A and B have previously and recently used a key, one party can transmit the new key to the other, encrypted using the old key
4. If A and b each has an encrypted connection to a third party C, C can deliver a key on the encrypted links to A and B



The Use of a Key Hierarchy

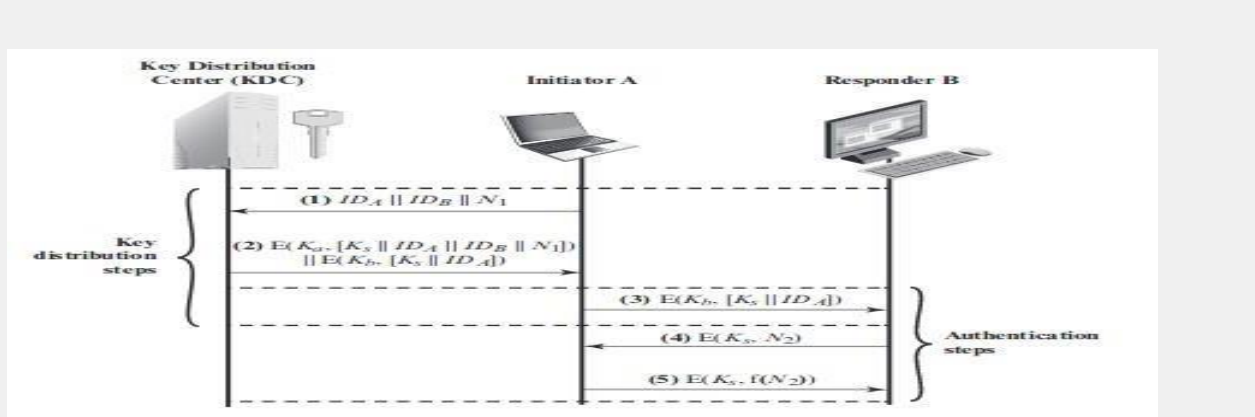
For end-to-end encryption some variation on option 4 has been widely adopted. In this scheme, a key distribution center responsible for distributing keys to pairs of users(hosts, processes, applications) as needed. Each user must share a unique key with the distribution center for purposes of key distribution.

The use of a key distribution center is based on the use of a hierarchy of keys. At a minimum, two levels of keys are used. Communication between end systems is encrypted using a temporary key, often referred to as a **session key.**

A Key Distribution Scenario

The key distribution concept can be deployed in a number of ways. A typical scenario is illustrated in Figure below.

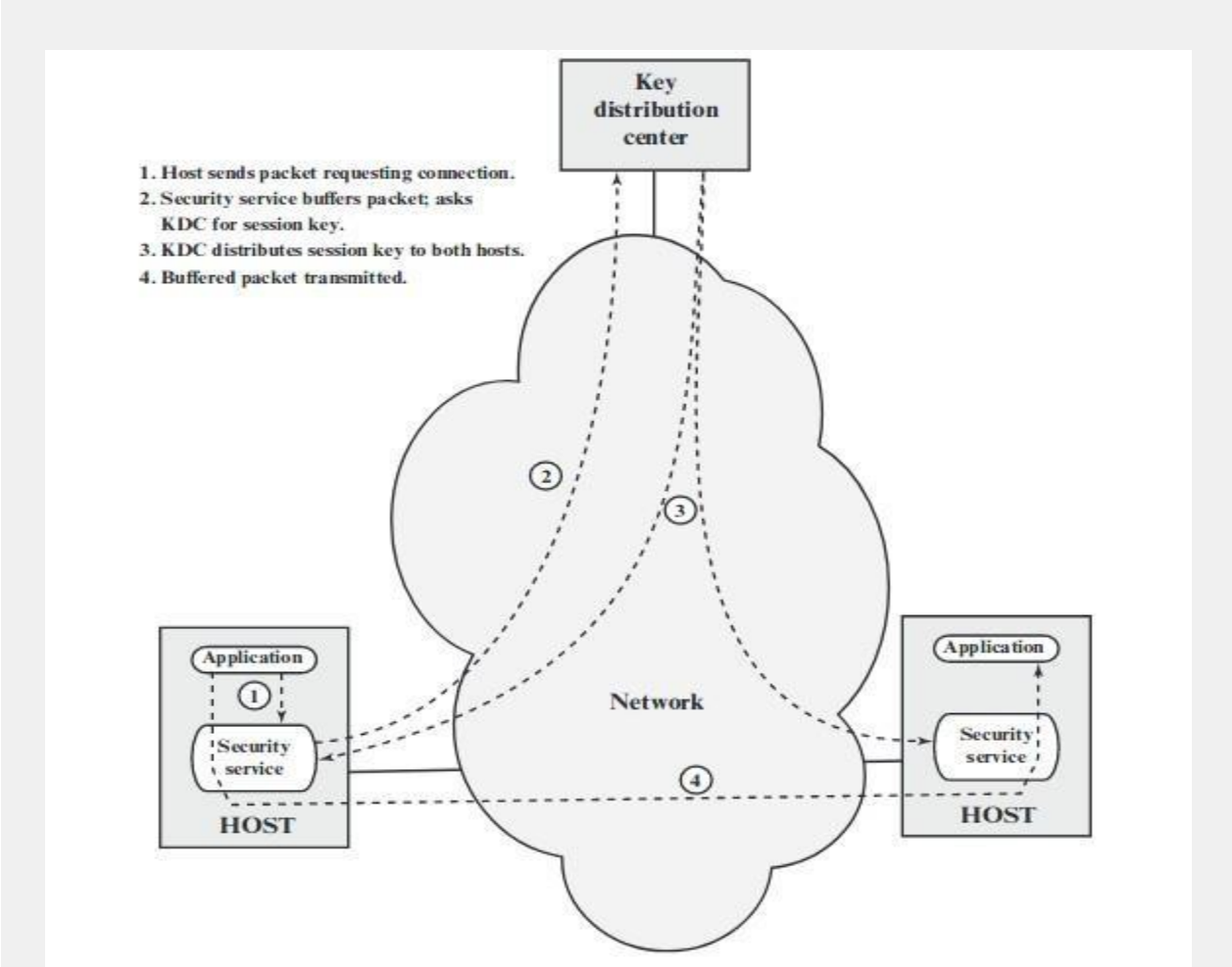
Let us assume that user A wishes to establish a logical connection with B and requires a one-time session key to protect the data transmitted over the connection.



Key Distribution Scenario

A Transparent Key Control Scheme

The scheme (Figure below) is useful for providing end-to-end encryption at a network or transport level in a way that is transparent to end users. The approach assumes that communication makes use of a connection-oriented end-to-end protocol, such as TCP.



Automatic Key Distribution for Connection-Oriented Protocol

Controlling Key Usage

The concept of key hierarchy and the use of automated key distribution techniques greatly reduce the number of keys that must be manually managed and distributed. For example, in addition to separating master keys from session keys, we may wish to define different types of session keys on the basis of use, such as:

- Data-encypting key, for general communication across a network
- PIN-encrypting key, for personal identification numbers(PINs) used in electronic funds transfer and point-of-sale applications.
- File-encrypting key, for encrypting files stored in publicly accessible locations

Symmetric key distribution using asymmetric encryption:

Because of the inefficiency of public-key cryptosystems, they are almost never used for the direct encryption if sizable blocks of data, but are limited to relatively small blocks. One of the most important uses of a public-key cryptosystem is to **encrypt secret keys for distribution**.

LECTURE-3

Distribution of public keys

Several techniques have been proposed for the distribution of public keys. Virtually all these proposals can be grouped into the following general schemas.

- **Public announcement**
- **Publicly available directory**
- **Public-key authority**
- **Public-key certificates**

Public Announcement of Public Keys

On the face of it, the point of public-key is that the public-key encryption is that the public key is public.

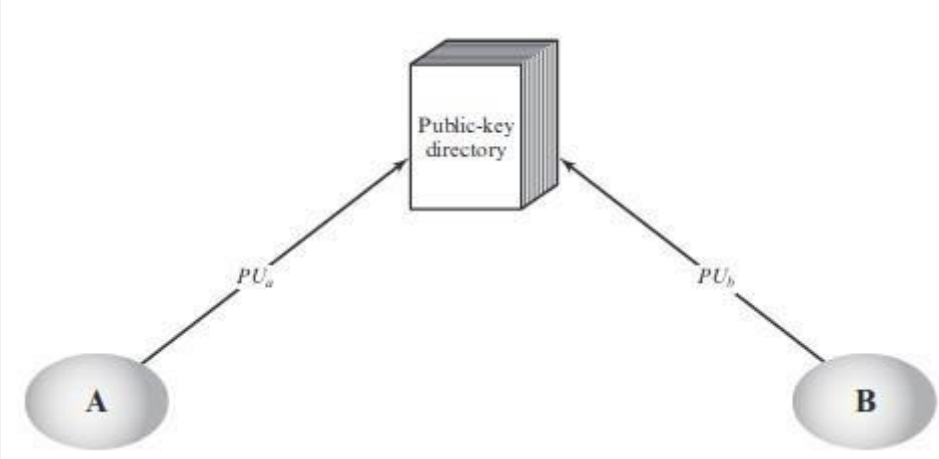
Although this approach is convenient, it has a major weakness. Anyone can forge such a public announcement. That is, some user could pretend to be user A and send a public key to another participant or broadcast such a public key. Until such time as user A discovers the forgery and alerts other participants, the forger is able to read all encrypted messages intended for A and can user forged keys for authentication.



Uncontrolled Public-Key Distribution

Public Available Directory

A greater degree of security can be achieved by maintaining a public available dynamic directory of public keys. Maintenance and distribution of the public directory would have to be responsibly of some trusted entity or organization.



Public-Key Publication

This scheme is clearly more secure than individual public announcements but still has vulnerabilities. If an adversary succeeds in obtaining or computing the private key of the directory authority.

Public-Key Authority

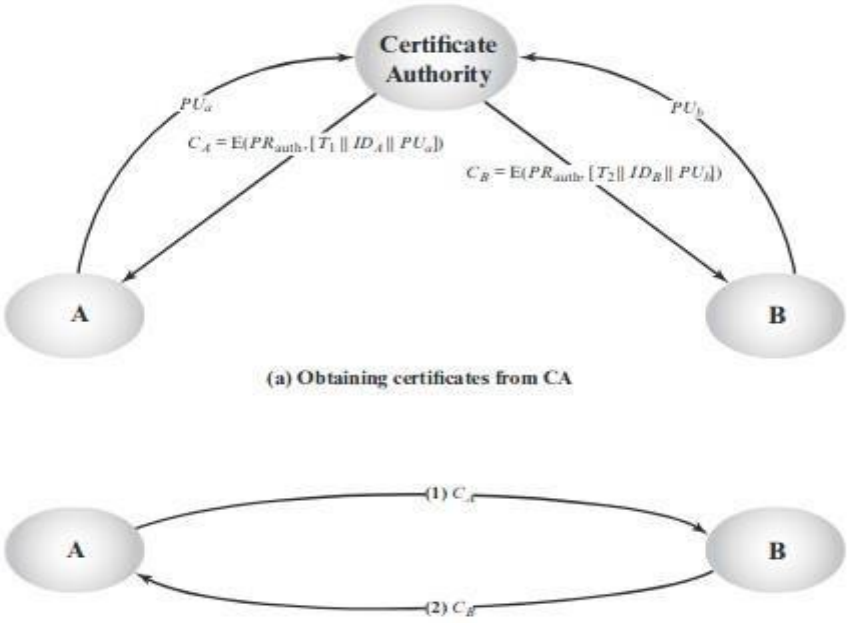
Stronger security for public-key distribution can be achieved by providing tighter control over the distribution of public keys from the directory. As before, the scenario assumes that a central authority maintains a dynamic directory of public keys of all participants. In a addition, each participant reliably knows a public key for authority, with only the authority knowing the corresponding private key.

Public-Key Certificates

An alternative approach, first suggested by Knohnfelder, is to use **certificates** that can be used by participants to exchange keys without contacting a public-key authority, in a way that is as reliable as if the keys were obtained directly from public-key authority. In essence, a certificate consists of a public key, an identifier of the key owner, and the whole block signed by a trusted third party. Typically, the third party is a certificate authority, such as a government agency or a financial institution, that is trusted by the user community.

A user can present his or her public key to the authority in a secure manner and obtain a certificate. The user can then publish the certificate. Anyone needing this user's public key can obtain the certificate and verify that it is valid by way of the attached trusted signature. A participant can also convey its key information to another by transmitting its certificate. Other participants can verify that the certificate was created by the authority. We can place the following requirements on the scheme:

1. Any participant can read a certificate to determine the name and public key of the certificate's owner
2. Any participant can verify that the certificate from the certificate authority and is not counterfeit
3. Only the certificate authority can create and update certificates
4. Any participant can verify the time validity of the certificate



Exchange of Public-Key Certificates

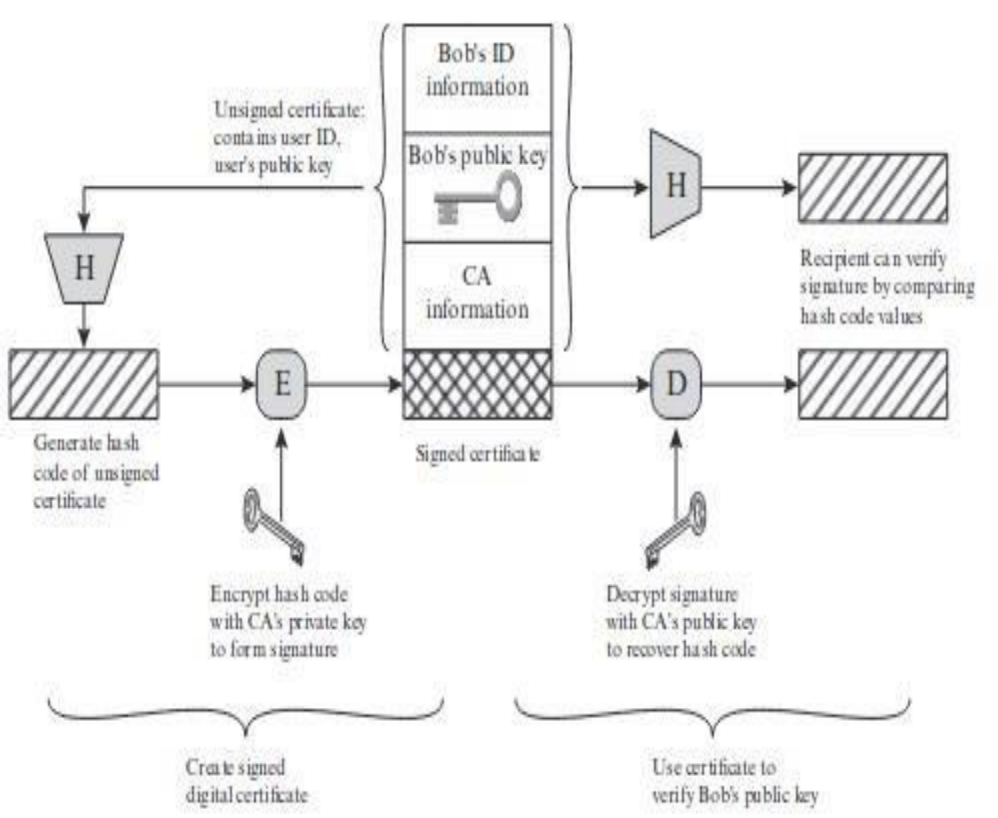
4. X.509 certificates

X.509 is part of the X.500 series of recommendations that define a directory service. The directory is, in effect, a server or distributed set of servers that maintains a database of information about users. The information includes a mapping from user name to network address, as well as other attributes and information about the users.

X.509 defines a framework for the provision of authentication services by the X500 directory to its users. In addition, X.509 defines alternative authentication protocols based on the use of public-key certificates.

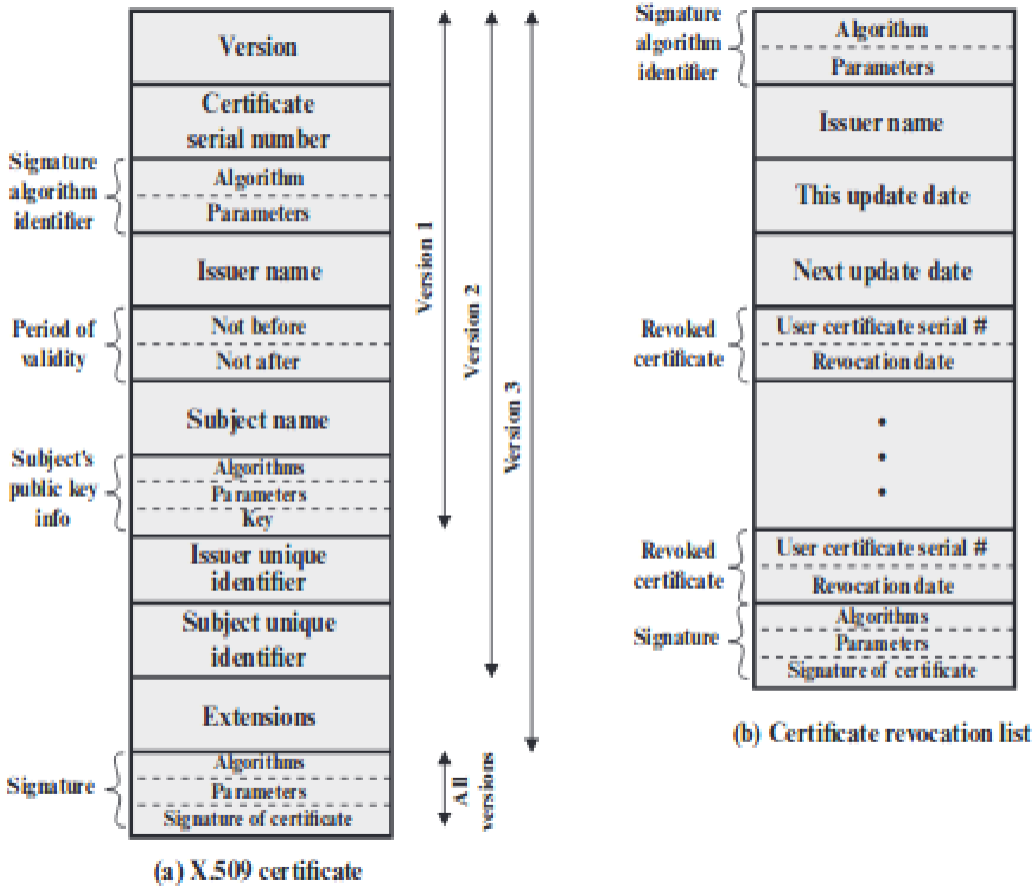
X.509 is an important standard because the certificate structure and authentication protocols defined in X.509 are used in variety of contexts. For example X.509 certificate format is used in S/MINE, IP security, and SSL/TLS.

X.509 is based on the use of public-key cryptography and digital signatures. The standard does not dictate the use of a specific digital signature algorithm nor a specific hash function.



X.509 Public-Key Certificate Use

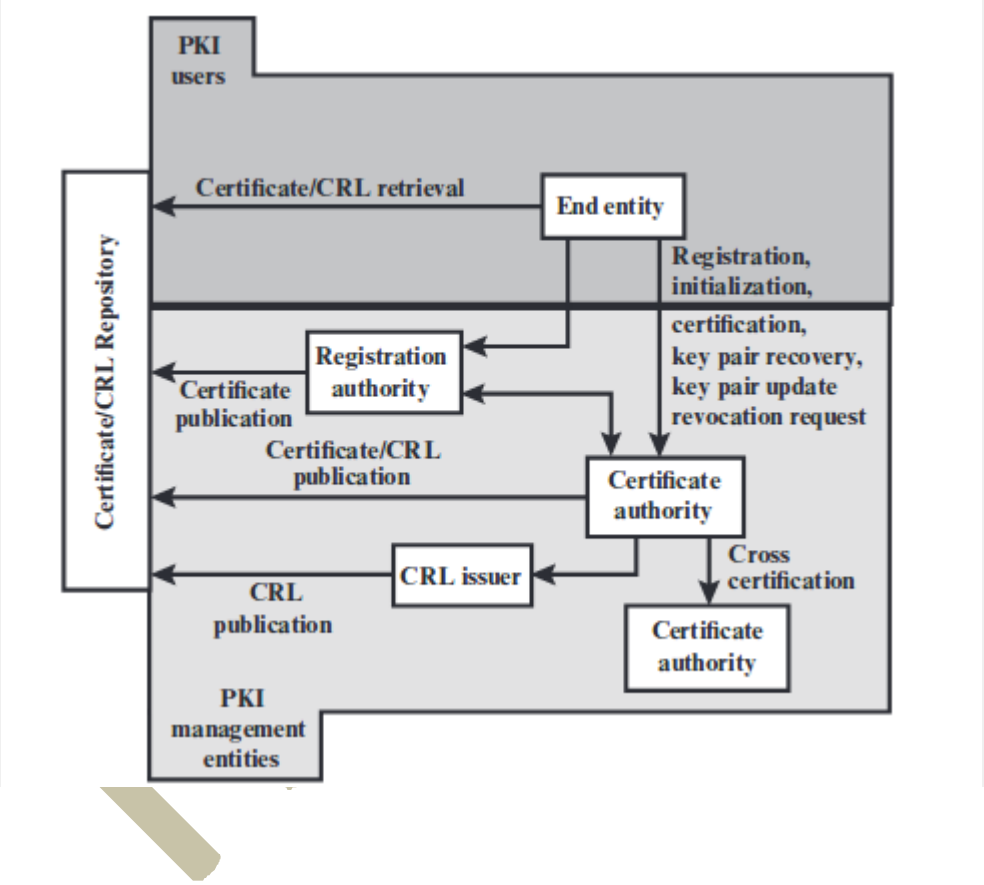
Certificates



X.509 formats

5. Public-key infrastructure

RFC 4949 defines Public-key infrastructure(PKI) as the set of hardware, software, people, policies, and procedures needed to create, manage, store, distribute, and revoke digital certificates based on asymmetric cryptography. The principal objective for developing a PKI is to enable secure, convenient, and efficient acquisition of public keys.



LECTURE-4

Public key infrastructure

The most distinct feature of Public Key Infrastructure (PKI) is that it uses a pair of keys to achieve the underlying security service. The key pair comprises of private key and public key.

Since the public keys are in open domain, they are likely to be abused. It is, thus, necessary to establish and maintain some kind of trusted infrastructure to manage these keys.

Key Management

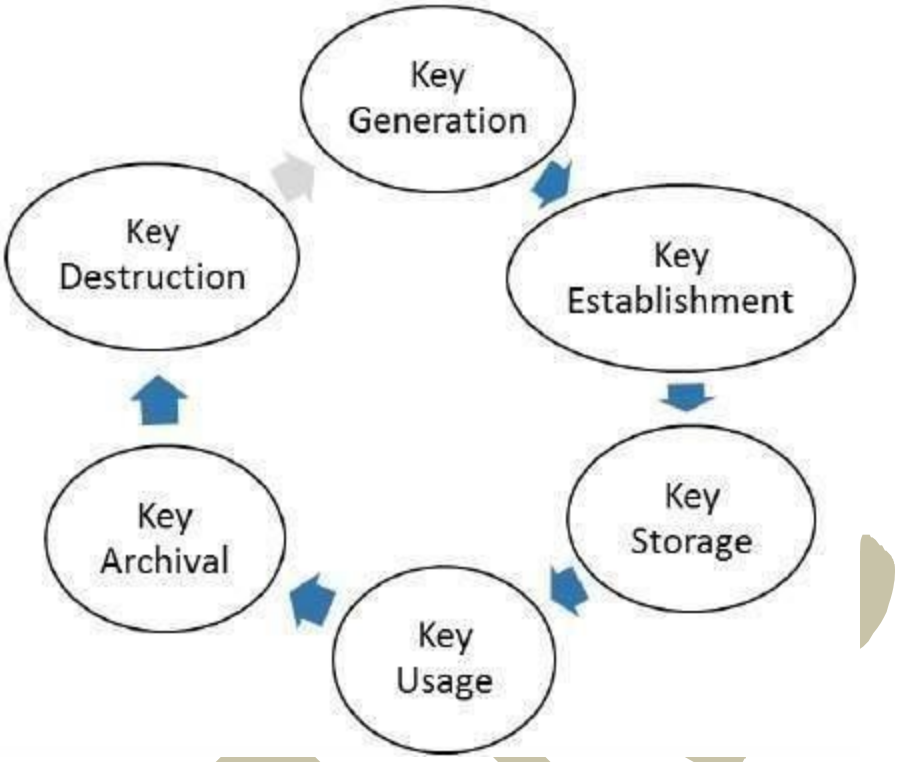
It goes without saying that the security of any cryptosystem depends upon how securely its keys are managed. Without secure procedures for the handling of cryptographic keys, the benefits of the use of strong cryptographic schemes are potentially lost.

It is observed that cryptographic schemes are rarely compromised through weaknesses in their design. However, they are often compromised through poor key management.

There are some important aspects of key management which are as follows −

- Cryptographic keys are nothing but special pieces of data. Key management refers to the secure administration of cryptographic keys.
- Key management deals with entire key lifecycle as depicted in the following illustration −

MITRC



- There are two specific requirements of key management for public key cryptography.
    - **Secrecy of private keys.** Throughout the key lifecycle, secret keys must remain secret from all parties except those who are owner and are authorized to use them.
    - **Assurance of public keys.** In public key cryptography, the public keys are in open domain and seen as public pieces of data. By default there are no assurances of whether a public key is correct, with whom it can be associated, or what it can be used for. Thus key management of public keys needs to focus much more explicitly on assurance of purpose of public keys.

The most crucial requirement of 'assurance of public key' can be achieved through the public-key infrastructure (PKI), a key management systems for supporting public-key cryptography.

Public Key Infrastructure (PKI)

PKI provides assurance of public key. It provides the identification of public keys and their distribution. An anatomy of PKI comprises of the following components.

- Public Key Certificate, commonly referred to as 'digital certificate'.
- Private Key tokens.
- Certification Authority.
- Registration Authority.
- Certificate Management System.

Digital Certificate

For analogy, a certificate can be considered as the ID card issued to the person. People use ID cards such as a driver's license, passport to prove their identity. A digital certificate does the same basic thing in the electronic world, but with one difference.

Digital Certificates are not only issued to people but they can be issued to computers, software packages or anything else that need to prove the identity in the electronic world.

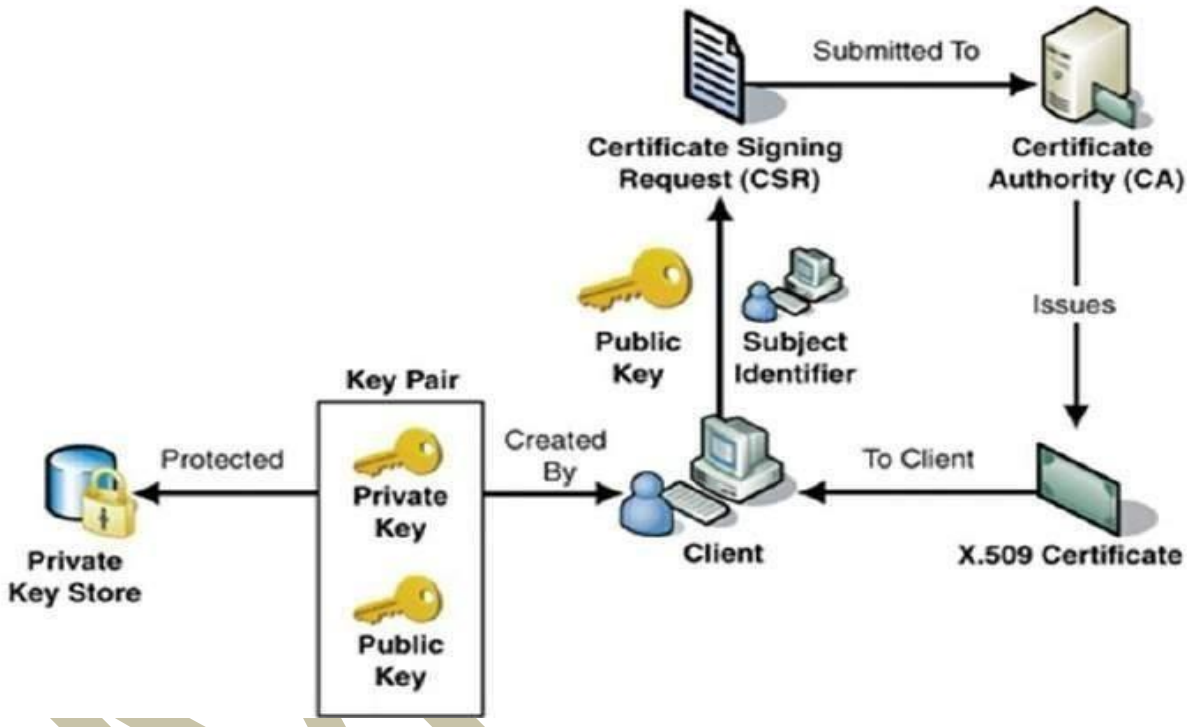
- Digital certificates are based on the ITU standard X.509 which defines a standard certificate format for public key certificates and certification validation. Hence digital certificates are sometimes also referred to as X.509 certificates.

  Public key pertaining to the user client is stored in digital certificates by The Certification Authority (CA) along with other relevant information such as client information, expiration date, usage, issuer etc.

- CA digitally signs this entire information and includes digital signature in the certificate.

- Anyone who needs the assurance about the public key and associated information of client, he carries out the signature validation process using CA's public key. Successful validation assures that the public key given in the certificate belongs to the person whose details are given in the certificate.

The process of obtaining Digital Certificate by a person/entity is depicted in the following illustration.



As shown in the illustration, the CA accepts the application from a client to certify his public key. The CA, after duly verifying identity of client, issues a digital certificate to that client.

Certifying Authority (CA)

As discussed above, the CA issues certificate to a client and assist other users to verify the certificate. The CA takes responsibility for identifying correctly the identity of the client asking for a certificate to be issued, and ensures that the information contained within the certificate is correct and digitally signs it.

Key Functions of CA

The key functions of a CA are as follows −

- **Generating key pairs** − The CA may generate a key pair independently or jointly with the client.
- **Issuing digital certificates** − The CA could be thought of as the PKI equivalent of a passport agency − the CA issues a certificate after client provides the credentials to confirm his identity. The CA then signs the certificate to prevent modification of the details contained in the certificate.
- **Publishing Certificates** − The CA need to publish certificates so that users can find them. There are two ways of achieving this. One is to publish certificates in the equivalent of an electronic telephone directory. The other is to send your certificate out to those people you think might need it by one means or another.
- **Verifying Certificates** − The CA makes its public key available in environment to assist verification of his signature on clients' digital certificate.
- **Revocation of Certificates** − At times, CA revokes the certificate issued due to some reason such as compromise of private key by user or loss of trust in the client. After revocation, CA maintains the list of all revoked certificate that is available to the environment.

Classes of Certificates

There are four typical classes of certificate −

- **Class 1** − These certificates can be easily acquired by supplying an email address.
- **Class 2** − These certificates require additional personal information to be supplied.

- **Class 3** – These certificates can only be purchased after checks have been made about the requestor's identity.
- **Class 4** – They may be used by governments and financial organizations needing very high levels of trust.

Registration Authority (RA)

CA may use a third-party Registration Authority (RA) to perform the necessary checks on the person or company requesting the certificate to confirm their identity. The RA may appear to the client as a CA, but they do not actually sign the certificate that is issued.

Certificate Management System (CMS)

It is the management system through which certificates are published, temporarily or permanently suspended, renewed, or revoked. Certificate management systems do not normally delete certificates because it may be necessary to prove their status at a point in time, perhaps for legal reasons. A CA along with associated RA runs certificate management systems to be able to track their responsibilities and liabilities.

Private Key Tokens

While the public key of a client is stored on the certificate, the associated secret private key can be stored on the key owner's computer. This method is generally not adopted. If an attacker gains access to the computer, he can easily gain access to private key. For this reason, a private key is stored on secure removable storage token access to which is protected through a password.

Different vendors often use different and sometimes proprietary storage formats for storing keys. For example, Entrust uses the proprietary .epf format, while Verisign, GlobalSign, and Baltimore use the standard .p12 format.

Hierarchy of CA

With vast networks and requirements of global communications, it is practically not feasible to have only one trusted CA from whom all users obtain their certificates. Secondly, availability of only one CA may lead to difficulties if CA is compromised.

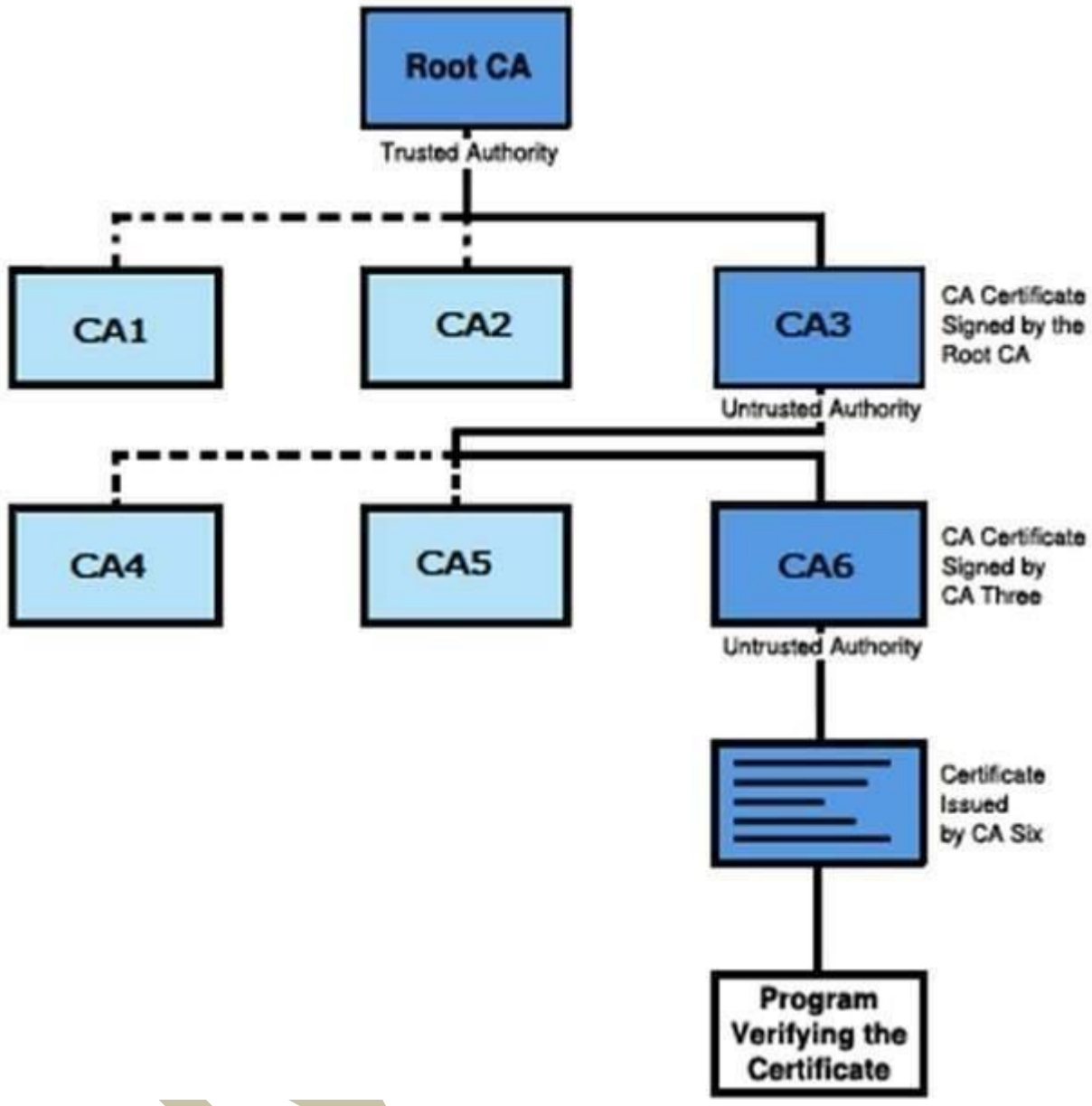
In such case, the hierarchical certification model is of interest since it allows public key certificates to be used in environments where two communicating parties do not have trust relationships with the same CA.

- The root CA is at the top of the CA hierarchy and the root CA's certificate is a self-signed certificate.
- The CAs, which are directly subordinate to the root CA (For example, CA1 and CA2) have CA certificates that are signed by the root CA.
- The CAs under the subordinate CAs in the hierarchy (For example, CA5 and CA6) have their CA certificates signed by the higher-level subordinate CAs.

Certificate authority (CA) hierarchies are reflected in certificate chains. A certificate chain traces a path of certificates from a branch in the hierarchy to the root of the hierarchy.

The following illustration shows a CA hierarchy with a certificate chain leading from an entity certificate through two subordinate CA certificates (CA6 and CA3) to the CA certificate for the root CA.

MITRC



Verifying a certificate chain is the process of ensuring that a specific certificate chain is valid, correctly signed, and trustworthy. The following procedure verifies a certificate chain, beginning with the certificate that is presented for authentication −

- A client whose authenticity is being verified supplies his certificate, generally along with the chain of certificates up to Root CA.

- Verifier takes the certificate and validates by using public key of issuer. The issuer's public key is found in the issuer's certificate which is in the chain next to client's certificate.
- Now if the higher CA who has signed the issuer's certificate, is trusted by the verifier, verification is successful and stops here.
- Else, the issuer's certificate is verified in a similar manner as done for client in above steps. This process continues till either trusted CA is found in between or else it continues till Root CA.

MITRC

LECTURE-7

Web Security threats and approaches, SSL architecture and protocol:

Web threats definition

Web-based threats, or online threats, are a category of cybersecurity risks that may cause an undesirable event or action via the internet.

Web threats are made possible by end-user vulnerabilities, web service developers/operators, or web services themselves. Regardless of intent or cause, the consequences of a web threat may damage both individuals and organizations.

This term typically applies to — but is not limited to — network-based threats in the following categories:

- **Private network threats -** impact sub-networks connected to the wider global internet. Typical examples can include home Wi-Fi or ethernet networks, corporate intranets, and national intranets.
- **Host threats -** impact specific network host devices. The term *host* often refers to corporate endpoints and personal devices, such as mobile phones, tablets, and traditional computers.
- **Web server threats -** impact dedicated hardware and software that serve web infrastructure and services.

What are web threats?

Internet-based threats expose people and computer systems to harm online. A broad scope of dangers fits into this category, including well-known threats like phishing and computer viruses. However, other threats, like offline data theft, can also be considered part of this group.

Web threats are not limited to online activity but ultimately involve the internet at some stage for inflicted harm. While not all web threats are created deliberately, many are intended — or have the potential — to cause:

- **Access denial.** Prevention of entry to a computer and/or network services**.**
- **Access acquisition.** Unauthorized or unwanted entry into a private computer and/or network services.
- Unauthorized or unwanted use of computer and/or network services.
- Exposing private data without permission, such as photos, account credentials, and sensitive government information.
- Unauthorized or undesired changes to a computer and/or network services.

In recent years, the landscape of web threats has grown significantly. Technologies like smart devices and high-speed mobile networks have allowed for an always-connected vector of malware, fraud, and other complications. Also, web adoption in areas like communications and productivity via the Internet of Things (IoT) has outpaced user security awareness.

As we continue to rely more on the web for daily living, it will keep exponentially rising as an attractive attack option for malicious parties. Convenience and a lack of caution around web use are among the top concerns that continue to pose new risks to privacy and security.

While targets are typically computer-based, human victims ultimately experience the lasting effects of a web threat.

How do web threats work?

When a web threat arises, certain circumstances align to make it a point-of-concern.

Namely, there are a few basic components to any web threat:

1. **Threat motives** give an intentional threat agent a reason or goal to cause harm. Some threat agents don't act intentionally or act autonomously and may, therefore, be absent of motive.
2. **Threat agents** are anything or anyone that can negatively impact — with the internet either as a threat vector or a target itself.
3. **Vulnerabilities** include any human behavior weakness, technology systems, or other resources that can lead to a damaging exploit or incident.
4. **Threat outcomes** are the negative results of a threat agent acting against one or more vulnerabilities.

   As these components interact, a threat becomes an attack on computer systems. **Threat motives** can include any of the following: financial, surveillance, information, retaliation, sabotage, and more.

   Threat agents are typically people with malicious intent. By extension, agents may also be anything that is manipulated into acting in favor of the original threat agent. However, some threat agents
   — such as destructive nature events — act entirely without human intervention.

The types of threat agents include:

- **Non-human agents:** Examples include malicious code (viruses, malware, worms, scripts), natural disasters (weather, geological), utility failure (electrical, telecom), technology failure (hardware, software), and physical hazards (heat, water, impact).
- **Intentional human agents:** Based on malicious intent. Can be internal (employees, contractors, family, friends, acquaintances) and external (professional and amateur hackers, nation-state actors and agencies, competitor corporations)
- **Accidental human agents:** Based on human error. Similar to intentional threats, this type can include internal and external agents.

- **Negligence-based human agents:** Based on careless behaviors or safety oversights. Again, this category can also include internal and external agents.

  Vulnerabilities may be points of weakness where someone or something can be manipulated. Vulnerabilities can be considered a web threat and a concern that enables other threats. This area typically includes some form of human or technical weakness that can lead to penetration, misuse, or destruction of a system.

  Threat outcomes may lead to disclosed private info, deceived users, disrupted computer system use, or seized access privileges. Web threats often result in, but are not limited to, causing:
- **Reputation damage:** Loss of trust from clients and partners, search engine blacklisting, humiliation, defamation, etc.
- **Operations disruption:** Operational downtime, access denial to web-based services such as blogs or message boards, etc.
- **Theft:** Financial, identity, sensitive consumer data, etc.

  Cybercriminals will use almost any vulnerability within an operating system (OS) or an application to conduct an attack. However, most cybercriminals will develop web threats that deliberately target some of the most common operating systems/applications, including:
- **Java:** Because Java is installed on over 3 billion devices (that are running under various operating systems) exploits can be created to target specific Java vulnerabilities on several different platforms/operating systems.
- **Adobe Reader:** Although many attacks have targeted Adobe Reader, Adobe has implemented tools to protect the program against exploit activity. However, Adobe Reader is still a common target.
- **Windows and Internet Explorer:** Active exploits still target vulnerabilities that were detected as far back as 2010 – including MS10-042 in Windows Help and Support Center, and MS04-028, which is associated with incorrect handling of JPEG files.
- **Android:** Cybercriminals use exploits to gain root privileges. Then, they can achieve almost complete control over the targeted device.

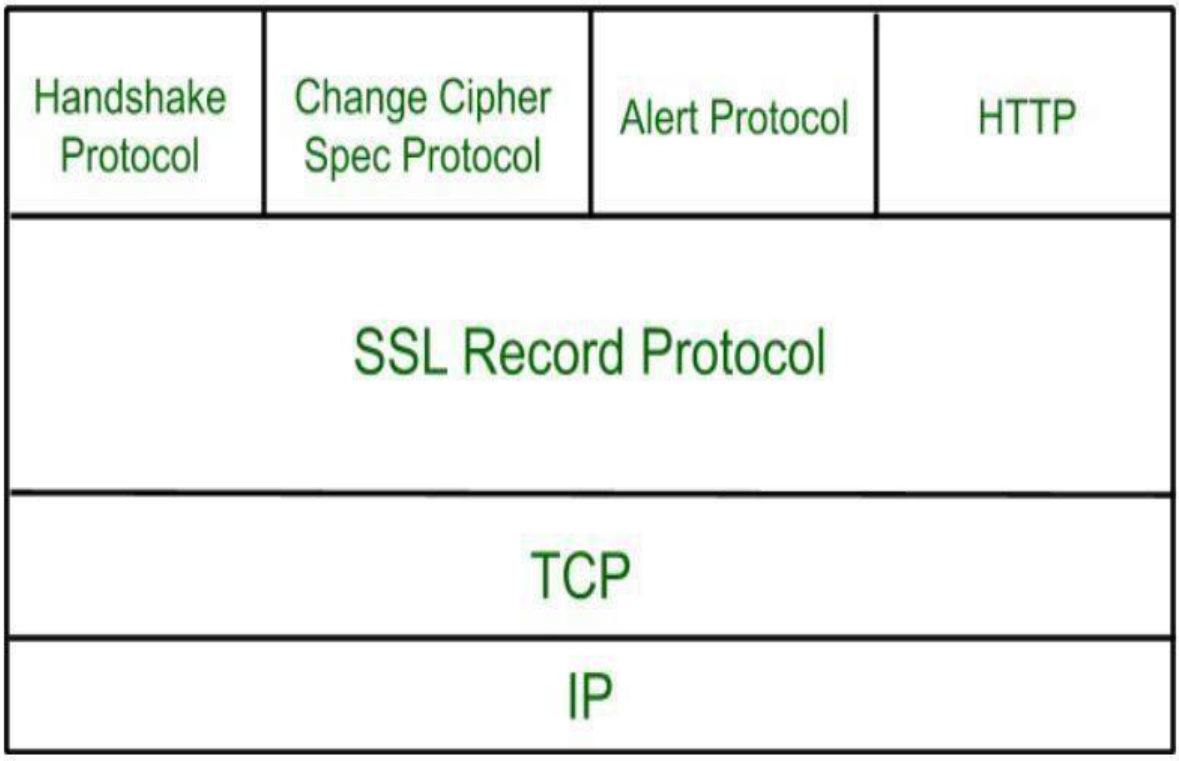
Secure Socket Layer (SSL)

Secure Socket Layer (SSL) provide security to the data that is transferred between web browser and server. SSL encrypt the link between a web server and a browser which ensures that all data passed between them remain private and free from attack.

Secure Socket Layer Protocols:

- SSL record protocol
- Handshake protocol
- Change-cipher spec protocol
- Alert protocol

SSL Protocol Stack:

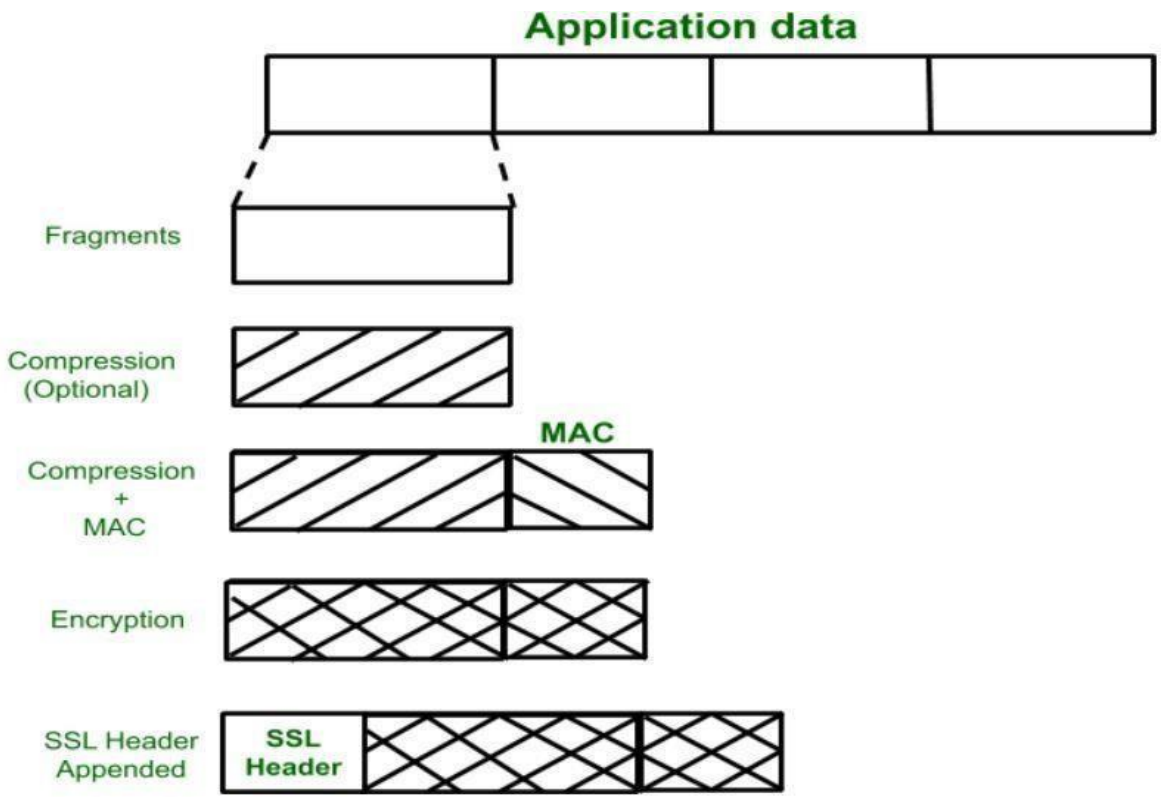


SSL Record Protocol:

SSL Record provide two services to SSL connection.

- Confidentiality
- Message Integerity

In SSL Record Protocol application data is divided into fragments. The fragment is compressed and then encrypted MAC (Message Authentication Code) generated by algorithms like SHA (Secure Hash Protocol) and MD5 (Message Digest) is appended. After that encryption of the data is done and in last SSL header is appended to the data.



HandshakeProtocol:

Handshake Protocol is used to establish sessions. This protocol allow client and server to authenticate each other by sending a series of messages to each other. Handshake protocol uses four phases to complete its cycle.

- **Phase-1:** In Phase-1 both Client and Server send hello-packets to each other. In this IP session, cipher suite and protocol version are exchanged for security purpose.
- **Phase-2:** Server send his certificate and Server-key-exchange. Server end the phase-2 by sending Server-hello-end packet.
- **Phase-3:** In this phase Client reply to the server by sending his certificate and Client-exchange-key.
- **Phase-4:** In Phase-4 Change-cipher suite occurred and after this Handshake Protocol ends.

Change-cipherProtocol:

This protocol uses SSL record protocol. Unless Handshake Protocol is completed, the SSL record Output will be in pending state. After handshake protocol the Pending state is converted into Current state. Change-cipher protocol consists of single message which is 1 byte in length and can have only one value. This protocol purpose is to cause the pending state to be copied into current state.



Alert Protocol:

This protocol is used to convey SSL-related alerts to the peer entity. Each message in this protocol contain 2 bytes.



Level is further classified into two parts:

- **Warning:**This Alert have no impact on the connection between sender and receiver.
- **Fatal Error:**
  This Alert breaks the connection between sender and receiver.

Silent Features of Secure Socket Layer:

- Advantage of this approach is that the service can be tailored to the specific needs of the given application.
- Secure Socket Layer was originated by Netscape.
- SSL is designed to make use of TCP to provide reliable end-to-end secure service.
- This is two-layered protocol.

LECTURE-8

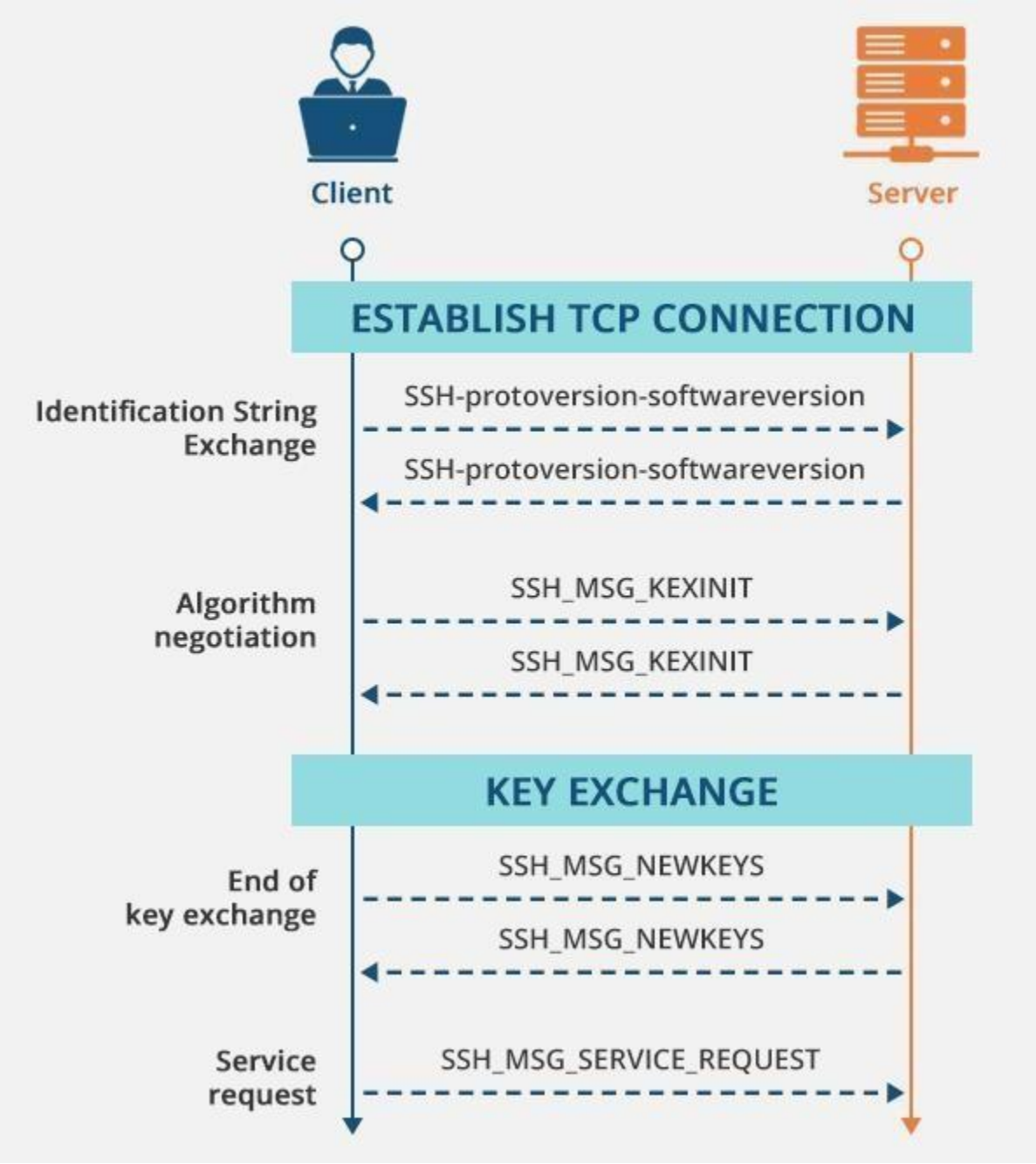
Transport layer security, HTTPS and SSH:

SSH and SSL/TLS are employing Asymmetric cryptography. These security protocols are intended to keep your personal information private during data transmission and secure communication over the Internet and public networks.
SSH (Secure Shell) and SSL/TLS (Secure Sockets Layer/ Transport Layer Security) are very similar as compare of security aspects – encryption, authentication and integrity processes. But when you are hunting for SSH vs SSL/TLS, here we are exploring similarities and differences between SSH vs SSL/TLS protocols.

What is SSH?

Secure shell, more commonly known as SSH, is a way to securely communicate with a remote computer. SSH is used for executing commands remotely by interacting with another system's operating shell. Originally created for UNIX-based computers, SSH is now easily available on Windows.

You will notice that SSH runs on TCP port 22. Other login methods in Putty — telnet and Rlogin, for instance — don't have the security that SSH does. The SSH protocol was created to replace insecure protocols for remote connections. SSH is a cryptographic protocol that creates a tunnel between two remote computers. Once the tunnel is established, the remote system shell is visible, and shell commands can be securely transmitted across the connection. Here is an illustration of an SSH handshake:

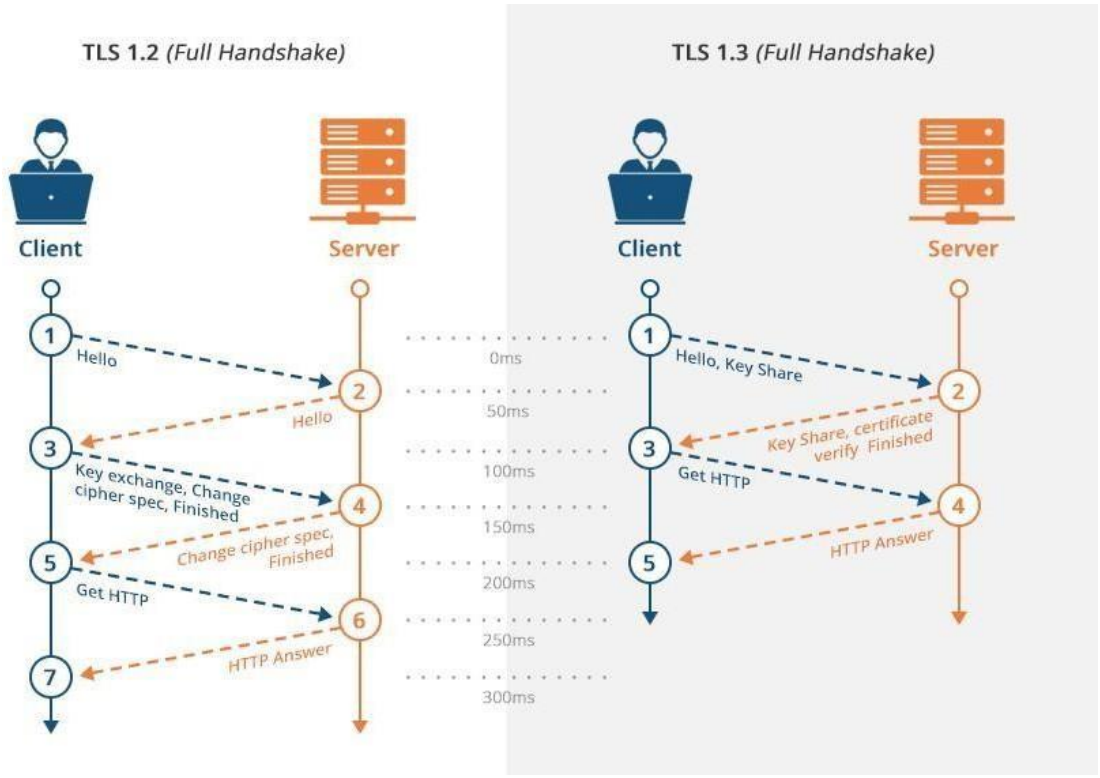

Client
Server
ESTABLISH TCP CONNECTION
Identification String Exchange
SSH-protoversion-softwareversion
SSH-protoversion-softwareversion
Algorithm negotiation
SSH_MSG_KEXINIT
SSH_MSG_KEXINIT
KEY EXCHANGE
End of key exchange
SSH_MSG_NEWKEYS
SSH_MSG_NEWKEYS
Service request
SSH_MSG_SERVICE_REQUEST

What is SSL/TLS?

We combine these two protocols as one because they do much the same thing. Both are still used, but TLS is gradually replacing SSL in network implementations. Like SSH, these are both cryptographic protocols. But the setup is different. An SSL/TLS connection uses the Public Key Infrastructure (PKI) and digital certificates to provide security and integrity to website transactions.

Secure Sockets Layer (SSL) and Transport Layer Security (TLS) are mechanisms for securing websites. While SSL 2.0 and 3.0 are obsolete version, they have been deprecated by IETF in favor of TLS (Some have suggested that TLS is an upgraded version of SSL 3.0). The difference is that TLS is more robust and secure against modern threats and vulnerabilities. If we look at TLS versions, then TLS 1.2 and 1.3 are recent versions that are adopted by major browser authorities. TLS 1.2 supports advanced encryption like ECC and AEAD cipher blocks while TLS 1.3 adds features of TLS 1.2 as well provides downgrade attack prevention.

SSL and TLS are handshake protocols. They are part of a server/client architecture that involves requests and responses to negotiate a connection between two computers. Here is an illustration of a TLS 1.3 handshake:



SSL and TLS use digital signatures generated by Certificate Authorities to enable a trust relationship between users and providers. Once the connection is established, it becomes possible to securely transmit sensitive information, such as might be passed between a bank and its customers. The standard TCP port used for Hypertext Transfer Protocol over SSL/TLS (HTTPS) is port 443.

SSH vs SSL/TLS – Differences Between both Security Protocols

SSH and SSL/TLS generally have different purposes. SSH is often used by network administrators for tasks that a normal internet user would never have to deal with. SSL/TLS, on the other hand, is used by the average internet user all the time. Any time someone uses a website with a URL that starts with HTTPS, he is on a site with SSL/TLS.

SSH (Secure Shell)	**SSL/TLS** (Secure Socket Layer/Transport Socket Layer)
SSH runs on port 22	SSL runs on port 443
SSH is for securely executing commands on a server.	SSL is used for securely communicating personal information.
SSH uses a username/password authentication system to establish a secure connection.	SSL normally uses X.509 digital certificates for server and client authentication.
SSH is working based on network tunnels.	SSL is working based on digital certificates.
SSH is a remote protocol	SSL is a security protocol
It is used to reduce security threats for remote server login	It allows secure transition of data between a server and the browser thus, keeps information intact.
SSH follows authentication process by server's verification done by client, session key generation, and client's authentication	SSL follows authentication process by exchange of digital certificate
Data integrity is measured with algorithms like SHA, SHA-2, SHA-256	Data integrity is measured with the message digest and added to encrypted data before the data is sent.

MITRC

Similarities Between SSH and SSL/TLS

It's understandable that someone might get SSH and SSL confused, since both are three-digit abbreviations that start with the same letter. But there are other similarities. First, these are both (and all three) protocols that are used in secure connections. Both use encryption to protect data that passes between two network devices.

The purpose of both types of protocols is to create a reliable connection. Without these two mechanisms, we would not have the security necessary to conduct business on the internet that we have today.

Conclusion

While there are other applications for these protocols, the basic differences are clear. SSH is generally a tool for technicians, and SSL/TLS is a mechanism for securing websites that is transparent to the user. Of course, these two are not mutually exclusive. SSH may use SSL/TLS as part of its secure solution. There are a variety of possible implementations for these versatile protocols. They are necessary these days with all the online threats that we face. It's another reminder that we all need to be careful and vigilant when using online resources.

LECTURE-5

Remote User-Authentication Principles

Remote User-Authentication Principles

- fundamental security building block
  - basis of access control & user accountability
- is the process of verifying an identity claimed by or for a system entity
- has two steps:
  - identification - specify identifier
  - verification - bind entity (person) and identifier
- distinct from message authentication

Means of User Authentication

There are four general means of authenticating a user's identity, which can be used alone or in combination

- **Something the individual knows**
  - Examples include a password, a personal identification number (PIN), or answers to a prearranged set of questions
- **Something the individual possesses**
  - Examples include cryptographic keys, electronic keycards, smart cards, and physical keys
  - This is referred to as a token
- **Something the individual is (static biometrics)**
  - Examples include recognition by fingerprint, retina, and face
- **Something the individual does (dynamic biometrics)**
  - Examples include recognition by voice pattern, handwriting characteristics, and typing rhythm

For network-based user authentication, the most important methods involve cryptographic keys and something the individual knows, such as a password

Mutual Authentication

- used to convince parties of each others identity and to exchange session keys
- may be one-way or mutual
- key issues are
  - confidentiality – to protect session keys
  - timeliness – to prevent replay attacks

Replay Attacks

1. The simplest replay attack is one in which the opponent simply copies a message and replays it later
2. An opponent can replay a timestamped message within the valid time window
3. An opponent can replay a timestamped message within the valid time window, but in addition, the opponent suppresses the original message; thus, the repetition cannot be detected
4. Another attack involves a backward replay without modification and is possible if symmetric encryption is used and the sender cannot easily recognize the difference between messages sent and messages received on the basis of content

Approaches to Coping With Replay Attacks

- Attach a sequence number to each message used in an authentication exchange
  - A new message is accepted only if its sequence number is in the proper order
  - Difficulty with this approach is that it requires each party to keep track of the last sequence number for each claimant it has dealt with
  - Generally not used for authentication and key exchange because of overhead

- Timestamps
  - Requires that clocks among the various participants be synchronized
  - Party A accepts a message as fresh only if the message contains a timestamp that, in A's judgment, is close enough to A's knowledge of current time

- Challenge/response
  - Party A, expecting a fresh message from B, first sends B a nonce (challenge) and requires that the subsequent message (response) received from B contain the correct nonce value

One-Way Authentication

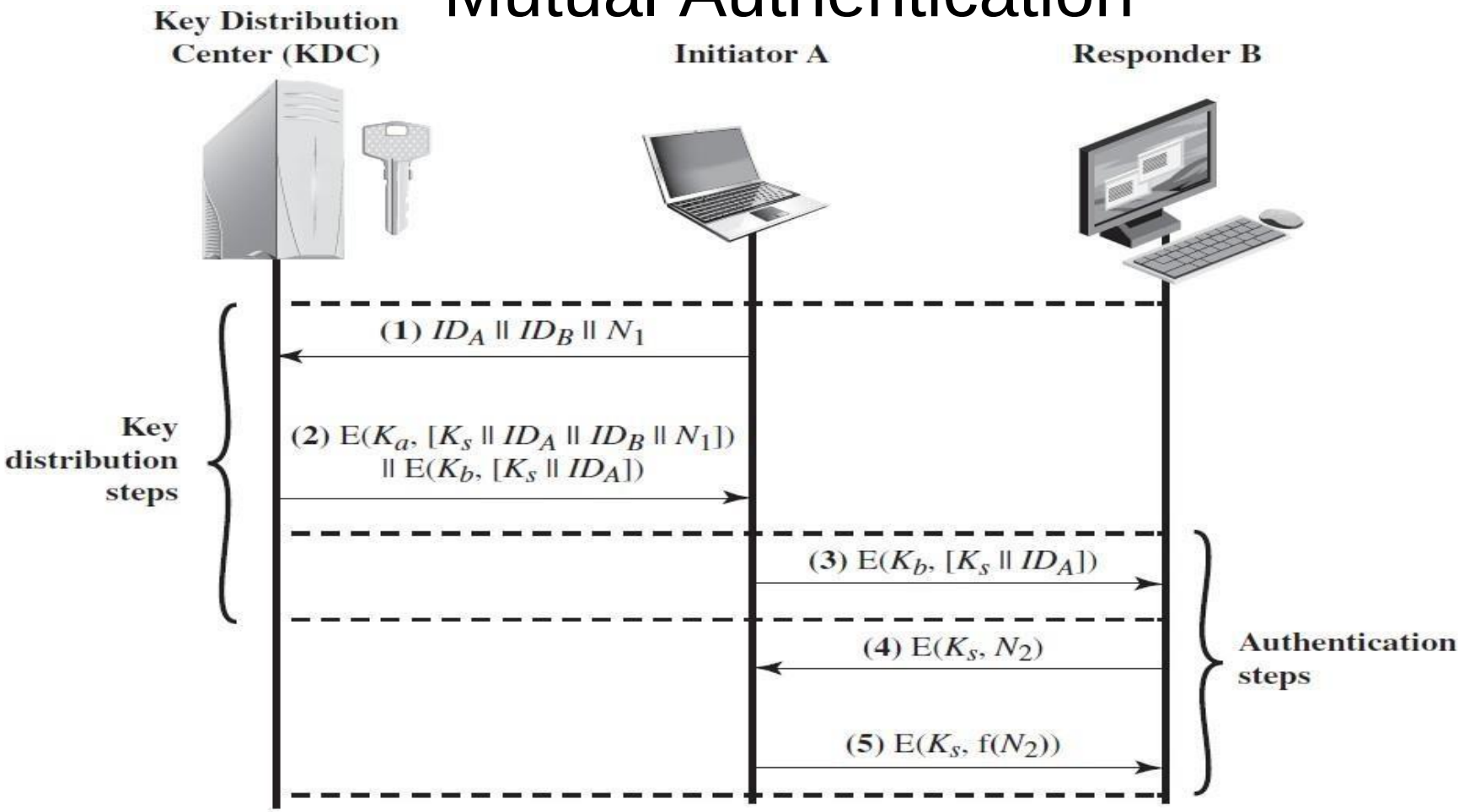
- required when sender & receiver are not in communications at same time (eg. email)
- have header in clear so can be delivered by email system
- may want contents of body protected & sender authenticated

Remote User-Authentication Using Symmetric Encryption

Mutual Authentication

- A two-level hierarchy of symmetric keys can be used to provide confidentiality for communication in a distributed environment
  - Strategy involves the use of a trusted key distribution center (KDC)
  - Each party shares a secret key, known as a master key, with the KDC
  - KDC is responsible for generating keys to be used for a short time over a connection between two parties and for distributing those keys using the master keys to protect the distribution

Mutual Authentication



1. $A \rightarrow KDC: \quad ID_A \| ID_B \| N_1$
2. $KDC \rightarrow A: \quad E(K_a, [K_s \| ID_B \| N_1 \| E(K_b, [K_s \| ID_A])])$
3. $A \rightarrow B: \quad E(K_b, [K_s \| ID_A])$
4. $B \rightarrow A: \quad E(K_s, N_2)$
5. $A \rightarrow B: \quad E(K_s, f(N_2))$

Mutual Authentication

1. $A \rightarrow KDC: \quad ID_A \| ID_B$
2. $KDC \rightarrow A: \quad E(K_a, [K_s \| ID_B \| T \| E(K_b, [K_s \| ID_A \| T])])$
3. $A \rightarrow B: \quad E(K_b, [K_s \| ID_A \| T])$
4. $B \rightarrow A: \quad E(K_s, N_1)$
5. $A \rightarrow B: \quad E(K_s, f(N_1))$

- Denning [DENN81, DENN82] proposes to overcome this weakness by a modification to the Needham/Schroeder protocol that includes the addition of a timestamp to steps 2 and 3
- T is a timestamp that assures A and B that the session key has only just been generated. Thus, both A and B know that the key distribution is a fresh exchange. A and B can verify timeliness by checking that

$$|\text{Clock} - T| < \Delta t_1 + \Delta t_2$$

Mutual Authentication

1. $A \rightarrow B: \quad ID_A \| N_a$
2. $B \rightarrow KDC: \quad ID_B \| N_b \| E(K_b, [ID_A \| N_a \| T_b])$
3. $KDC \rightarrow A: \quad E(K_a, [ID_B \| N_a \| K_s \| T_b]) \| E(K_b, [ID_A \| K_s \| T_b]) \| N_b$
4. $A \rightarrow B: \quad E(K_b, [ID_A \| K_s \| T_b]) \| E(K_s, N_b)$

- In [KEHN92], an attempt is made to respond to the concerns about suppressreplay attacks and at the same time fix the problems in the Needham/Schroeder protocol. Subsequently, an inconsistency in this latter protocol was noted and an improved strategy was presented in [NEUM93a]

Mutual Authentication

1. $A \rightarrow B: \quad E(K_b, [ID_A \| K_s \| T_b]) \| N'_a$
2. $B \rightarrow A: \quad N'_b \| E(K_s, N'_a)$
3. $A \rightarrow B: \quad E(K_s, N'_b)$

- This protocol provides an effective, secure means for A and B to establish a session with a secure session key

One-Way Authentication

1. $A \rightarrow KDC: \quad ID_A \| ID_B \| N_1$
2. $KDC \rightarrow A: \quad E(K_a, [K_s \| ID_B \| N_1 \| E(K_b, [K_s \| ID_A])])$
3. $A \rightarrow B: \quad E(K_b, [K_s \| ID_A]) \| E(K_s, M)$

- This scheme requires the sender to issue a request to the intended recipient, await a response that includes a session key, and only then send the message.

LECTURE-6

Kerberos

Kerberos

- Authentication service developed as part of Project Athena at MIT
- A workstation cannot be trusted to identify its users correctly to network services
  - A user may gain access to a particular workstation and pretend to be another user operating from that workstation
  - A user may alter the network address of a workstation so that the requests sent from the altered workstation appear to come from the impersonated workstation
  - A user may eavesdrop on exchanges and use a replay attack to gain entrance to a server or to disrupt operations

- Kerberos provides a centralized authentication server whose function is to authenticate users to servers and servers to users
  - Relies exclusively on symmetric encryption, making no use of public-key encryption

Kerberos Requirements

- its first report identified requirements as:
  - Secure : A network eavesdropper should not be able to obtain the necessary information to impersonate a user
  - Reliable : Should be highly reliable and should employ a distributed server architecture with one system able to back up another
  - Transparent : Ideally, the user should not be aware that authentication is taking place beyond the requirement to enter a password
  - Scalable : The system should be capable of supporting large numbers of clients and servers
- implemented using an authentication protocol based on Needham-Schroeder

Kerberos Version 4

- a basic third-party authentication scheme
- have an Authentication Server (AS)
  - users initially negotiate with AS to identify self
  - AS provides a non-corruptible authentication credential (ticket granting ticket TGT)
- have a Ticket Granting server (TGS)
  - users subsequently request access to other services from TGS on basis of users TGT
- using a complex protocol using DES

Overview of Kerberos

2. AS verifies user's access right in
databaspandcreaw:sCcketgrsnNngflcbn
andsesmonkey.Resuhsme€ncrypmd
uMngkcydsnvedf m 'Rid woFd.

oncsper
user logon
session



1. Us'er logs on to
wmksMUonand
requests service on host

request ticket-granting ticket

ticket + session key

request service-granting ticket

ticket + sessi

once per
type of service

Kerberos

NuHienticmtinn
server

Yicket-
granting
sezwez° (TCS)

3. Workstation prompts
umxfmpasswoMtodw t
mconñngrosssageandthen
BsndUcksCanO
aufhenncatorfha£conuñns
user's name, network
BO Hfls' and time to TGS.

4. TG3decrypts ticket and authenticator. verified reqElBst, and then creates ticket for requested application server.

request service

provide server authenticator

once per
Service session



Host/
application
server

5. WOfks'tation sends Lickot and autfienLicatoz to host.

6. Host verifies that ticket and authenticator match, and then grants access to s'eFViCR. If mutual authentication 15. *req* uir *ed* server returns an autbenticator.

Kerberos Version 5

- developed in mid 1990's
- specified as Internet standard RFC 1510
- provides improvements over v4
  - addresses environmental shortcomings
    - encryption alg, network protocol, byte order, ticket lifetime, authentication forwarding, interrealm auth
  - and technical deficiencies
    - double encryption, non-std mode of use, session keys, password attacks

- **Encryption system dependence:** In version 5, ciphertext is tagged with an encryption-type identifier so that any encryption technique may be used
- **Internet protocol dependence:** Version 5 network addresses are tagged with type and length, allowing any network address type to be used.
- **Message byte ordering:** In version 5, all message structures are defined using Abstract Syntax Notation One (ASN.1) and Basic Encoding Rules (BER), which provide an unambiguous byte ordering.
- **Ticket lifetime:**tickets include an explicit start time and end time, allowing tickets with arbitrary lifetimes.
- **Authentication forwarding:**Version 5 does not allow credentials issued to one client to be forwarded to some other host and used by some other client.
- **Interrealm authentication:**Version 5 supports a method that requires fewer relation
-

- **Double encryption:**tickets provided to clients are encrypted twice, The second encryption is not necessary and is computationally wasteful.
- **PCBC encryption:**Version 5 provides explicit integrity mechanisms, allowing the standard CBC mode to be used for encryption. In particular, a checksum or hash code is attached to the message prior to encryption using CBC.
- **Session keys:**In version 5, it is possible for a client and server to negotiate a subsession key, which is to be used only for that one connection. A new access by the client would result in the use of a new subsession key.
- **Password attacks:** versions are vulnerable to a password attack

Remote User Authentication Using Asymmetric Encryption

Mutual Authentication

- Public-key encryption for session key distribution
  - Assumes each of the two parties is in possession of the current public key of the other
  - May not be practical to require this assumption
- Denning protocol using timestamps
  - Uses an authentication server (AS) to provide public-key certificates
  - Requires the synchronization of clocks
- Woo and Lam makes use of nonces
  - Care needed to ensure no protocol flaws

One-Way Authentication

- Have public-key approaches for e-mail
  - Encryption of message for confidentiality, authentication, or both
  - The public-key algorithm must be applied once or twice to what may be a long message
- For confidentiality encrypt message with one-time secret key, public-key encrypted
- If authentication is the primary concern, a digital signature may suffice